COSC204
Computer Systems

6809 Assembly Language Programming

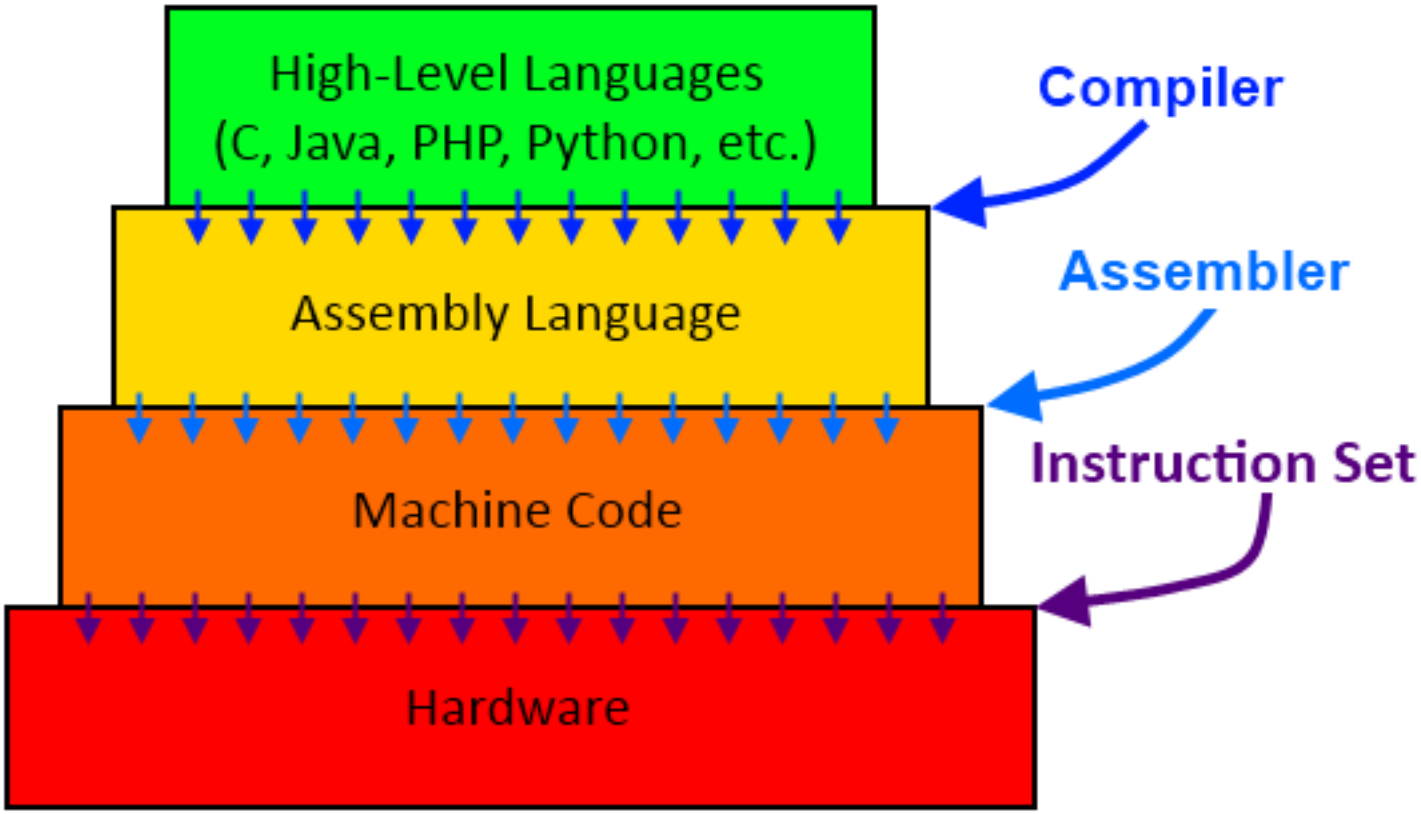
Overview

- Last Lecture
  - Architecture of a computer
- This Lecture
  - Machine code programming
  - The programmer's model
  - 6809 Assembly language
  - Source
    - Any book on 6809
      - Lance A. Leventhal (1981) 6809 Assembly Language Programming, Osborne McGraw-Hill, ISBN: 9780079310354
      - Card D. Warren (1980) The MC6809 Cookbook, Tab Books
- Next Lecture
  - Assembly language programming

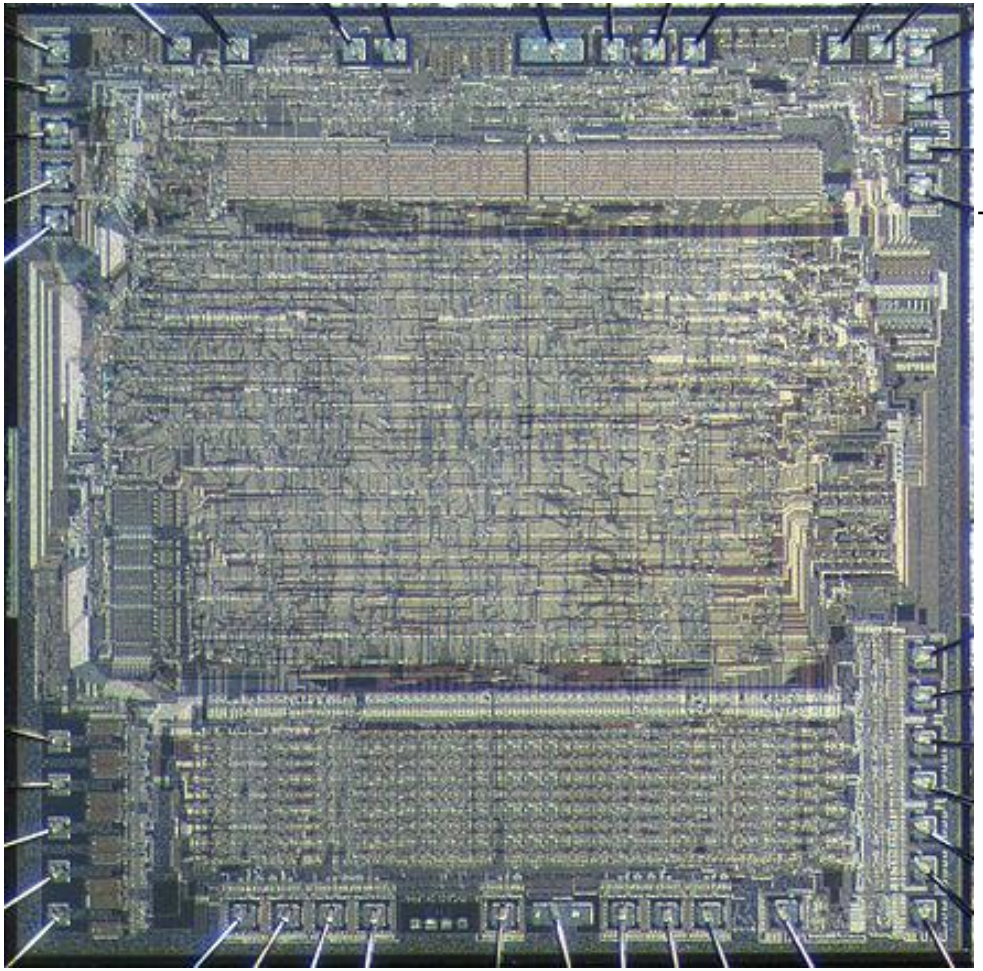
Warnings

- Different CPU architectures have their own machine codes and their own assembly languages
- Assembly language programs are ***not*** portable across CPU architectures (e.g. 6809 to x86 to ARM) but are often backwards compatible (e.g. x86 programs work on x86_64 family)

Working Up…


High-Level Languages
(C, Java, PHP, Python, etc.)
Compiler
Assembly Language
Assembler
Machine Code
Instruction Set
Hardware

Motorola MC6809 CPU



CPU	Year	Transistors
6809	1978	9,000
Apple M3 Ultra	2025	184,000,000,000

https://en.wikipedia.org/wiki/Transistor_count



https://en.wikipedia.org/wiki/Motorola_6809

TRS80, Commodore, Vectrex

- 1980: TRS-80 Color Computer (CoCo)
- 1981: Commodore SuperPET
- 1982: Vectrex







https://en.wikipedia.org/wiki/Motorola_6809

New Zealand Computers

- 1981: Amber Pegasus
- 1981: Poly, and Poly 2



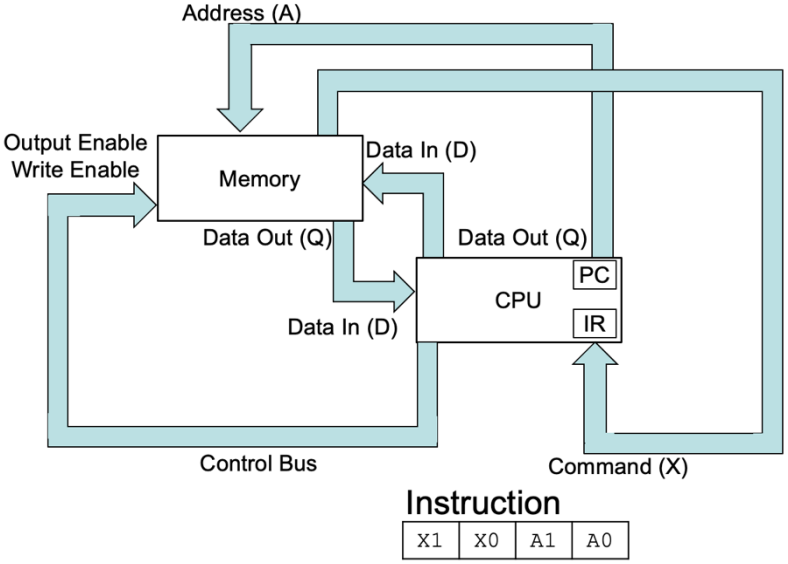
https://en.wikipedia.org/wiki/Aamber_Pegasus



https://en.wikipedia.org/wiki/Poly-1

Machine Code

- Recall that we programmed the computer by putting bit-patterns into it. Those bit patterns determined what the CPU did and to which memory location



- We call programming in 1s and 0s in this way ***machine code programming***

Fibonacci in 6502 Machine Code

- Machine code programs look like this:

```
4C 13 00 00 00 00 00 00
00 00 00 00 00 00 00 00
00 00 00 A2 10 A9 01 8D
10 00 8D 11 00 8D 12 00
A9 31 8D 0F 00 8D 0F 00
AD 10 00 6D 11 00 8D 12
00 69 30 8D 0F 00 AD 11
00 8D 10 00 AD 12 00 8D
11 00 CA D0 E3
```

http://www.visual6502.org/JSSim/expert.html?loglevel=0&a=0000&d=4C13000000000000000000000000000000A210A9018D10008D11008D1200A9318D0F008D0F00AD10006D11008D120069308D0F00AD11008D1000AD12008D1100CAD0E3

Machine Code

- These numbers tell the computer what to do

- They tell the CPU to
  - Assign values to registers
  - Load registers from memory
  - Add numbers to registers
  - Store registers in memory
  - And so on

Machine Code

- It is very difficult to program using numbers
- Programs are slow to enter
  - Each individual bit must be chosen by the programmer
- The programs are not human readable
- Programs are tiresome to write
- Debugging is extremely difficult

It Can Be Easier

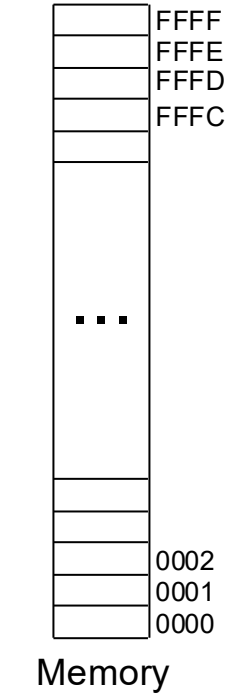
- What if we give mnemonics (names) to the numbers
  - Then we can program symbolically
  - And its easier to remember names than numbers
- We call this ***assembly language programming***
- But how does the computer look to a programmer who is doing this?

Programmer's Model

- The programmer's model of a computer is not the same as the hardware model. The hardware makes the computer look a particular way to the programmer
  - For example, your computer might have several memory chips in it, but to the programmer it looks like one continuous block of memory

6809 Memory

- ***16-bit address bus*** and ***8-bit data bus***
  - That is, 64KB (2^{16} Bytes) of memory
    - The ***memory space*** or ***address space***
- Each memory location is numbered
  - From $0000 to $FFFF
  - We use '$' to mean hexadecimal (hex, base 16)
  - For our purposes, all memory locations exist



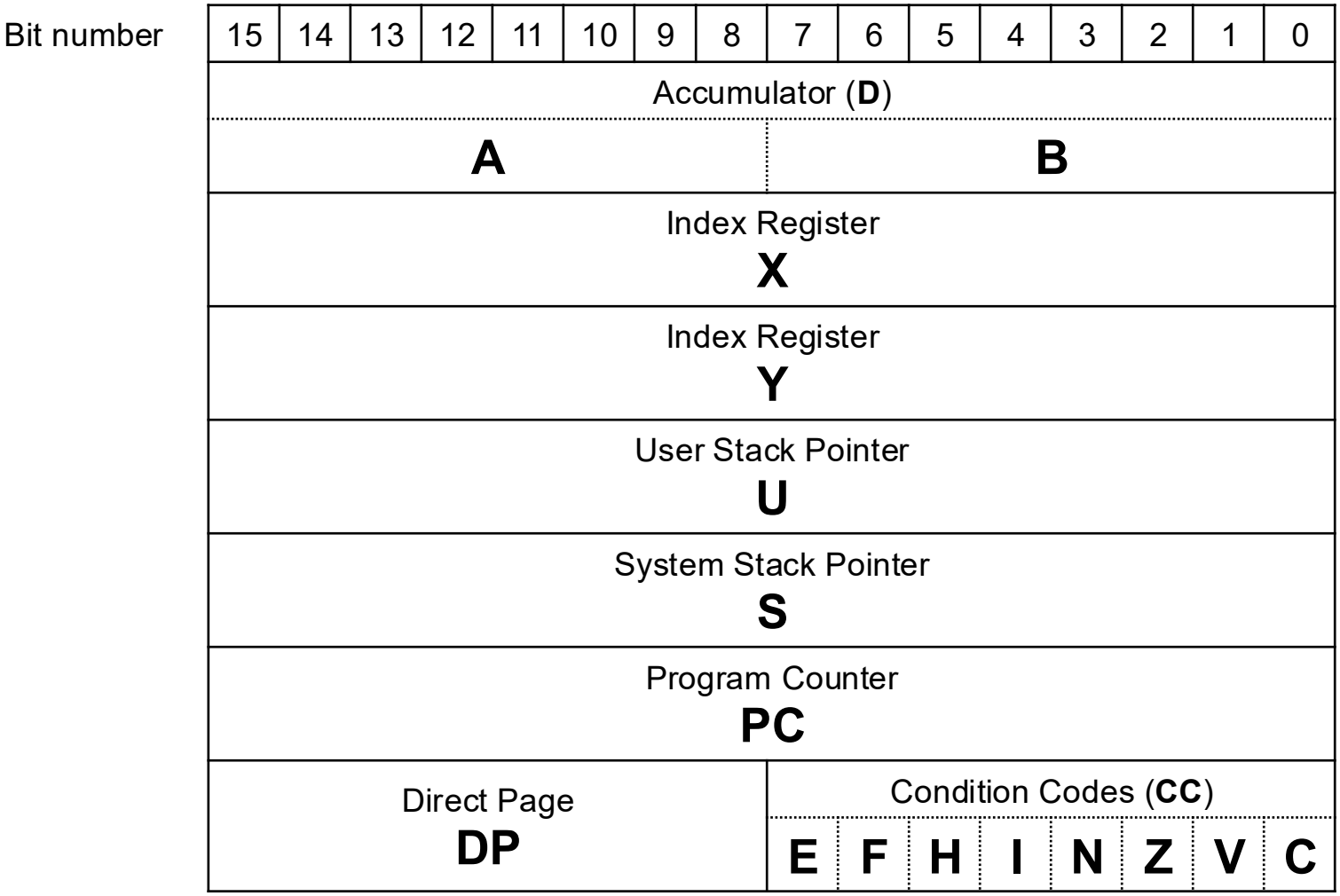
- That is, the computer memory is just an array

```
uint8_t memory[65536];
```

6809 Memory

- The 6809 memory space is ***conceptually*** broken into pages of 256 bytes each
- The first page is called zero-page (all addresses are of the form $00xx)
- The second page is called page 1 (all addresses are of the form $01xx)
- And so on

6809 Registers



X, Y, D=(AB)

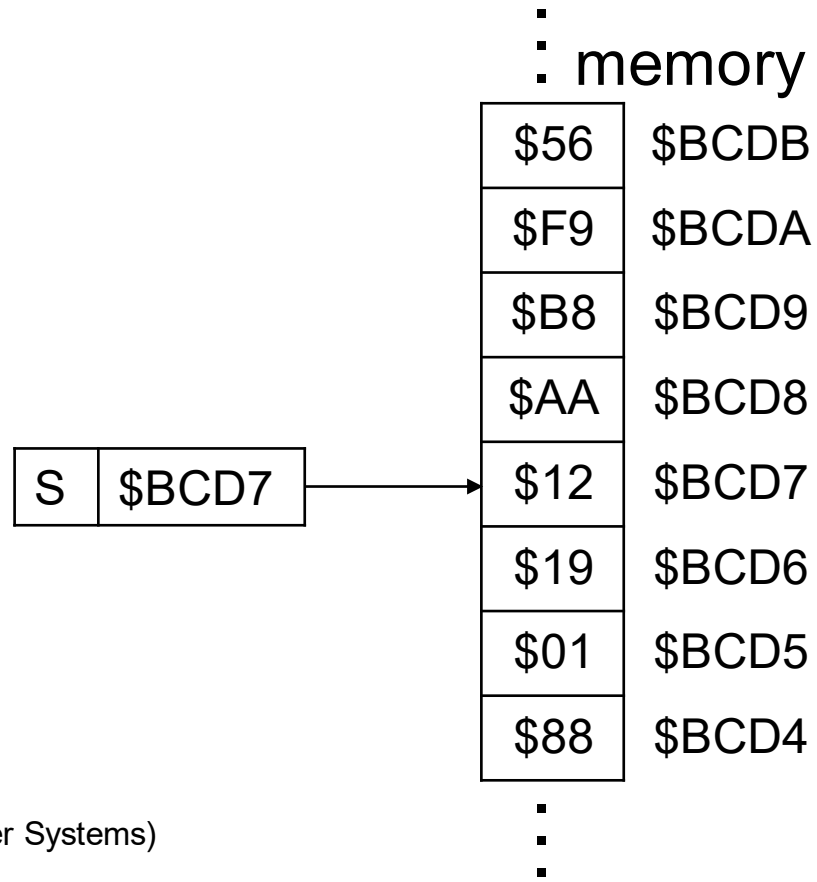
- Registers are just like ***global*** variables
  - But there are very few of them
- Some are general purpose
  - `X`, `Y`
    - Also used for indexing into that array we call memory
- Some are broken down like a `struct` or `union`
  - `D` is made up of `A` and `B`
  - Write to `A` or `B` and `D` changes
  - Write a 16-bit value to `D`
    - Read each 8-bit byte using `A` or `B`

DP and PC

- Some have special meaning
  - `DP`, the Direct Page register, can be used to make instructions that refer to the same memory page faster
    - We're going to (mostly) ignore this register
  - `PC`, the Program Counter, stores the location of the instruction that is currently being executed

The System Stack Pointer

- The System Stack Pointer, `S`, is just an index into the byte array we call memory



The System Stack Pointer

- We can say things like `LDS #$BCD7`

  `S=0xBCD7`

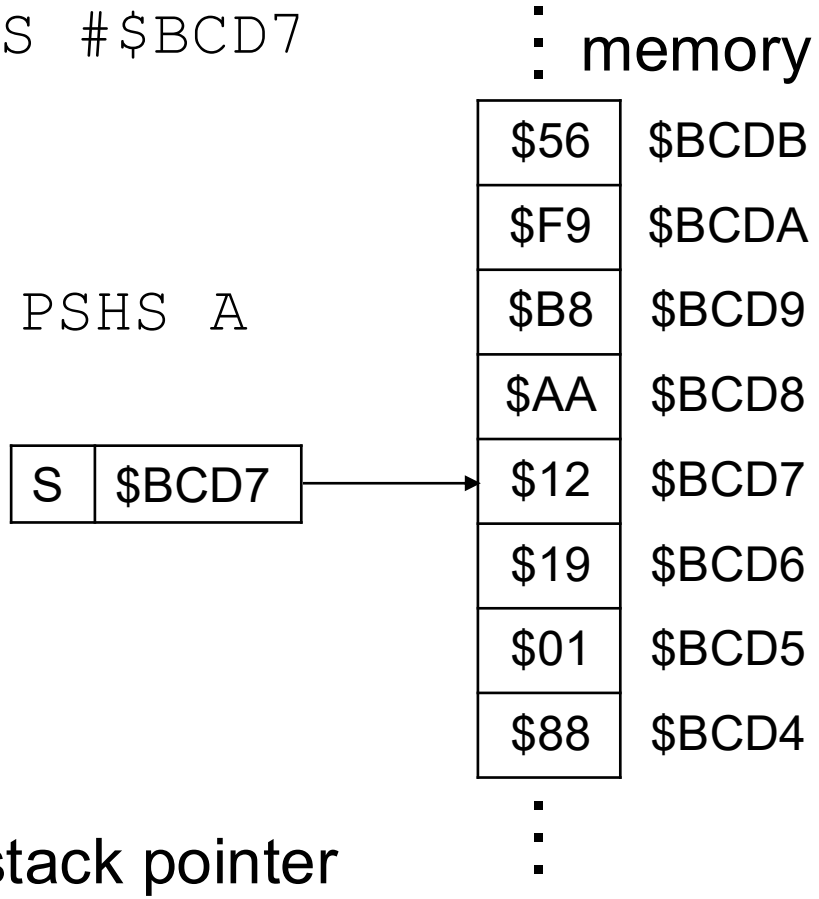
- But it's more useful to say `PSHS A`

  `memory[--S]=A`

- Or `PULS A`

  `A = memory[S++]`

- Which is why we call it a stack pointer



The User Stack Pointer

- `U`, the User Stack Pointer is just like `S`
  - EXCEPT: `S` is used for routine (function) calls too
    - See Lecture 7 on how routine calls use the system stack

6809 Condition Codes (CC)

- The Condition Codes (CC) holds information about the result of the previous operation
  - We already know about the ripple-carry-adder
    - Where does the final carry go? It goes into flag C
    - Where does the first carry come from? It comes from flag C

Bit number	7	6	5	4	3	2	1	0
	E	F	H	I	N	Z	V	C

} Flags

Flag	Meaning	Flag	Meaning
E	Entire (on interrupt)	N	Negative
F	FIRQ Mask (on 1)	Z	Zero
H	Half Carry	V	Overflow
I	IRQ Mask (on 1)	C	Carry

Instruction Set

- The set of all instructions for a particular CPU is called the ***instruction set***
  - It is determined by the CPU designers
- Instructions consist of one or more fields
  - The mnemonic ***opcode***
    - E.g. Return from subroutine (`RTS`)
  - And (optionally) parameters called ***operands***
    - E.g. GOTO (`BRA MORE`)
- Instructions are often of different lengths (or widths)
  - `RTS` is one byte on the 6809
  - `BRA MORE` is two bytes on the 6809
  - On ARM32 all instructions are 32-bits (4 bytes) wide

Instruction Syntax

- Normally case insensitive
- Syntax

  <label:> <opcode> <operands> <;comment>

- Labels
  - Start with a letter and ends with a colon (can be alphanumeric)
  - Some assembles don't require the colon
- Operands

  # immediate (if no #, then it's an index into the memory array)

  $ hex value

  % binary value

Examples

- Example
  - Load `A` with the ***value*** $0F
    - This is like: A = 0x0F

```
LDA #$0F
```

  - Load `A` with the value stored at ***memory location*** $000F
    - This is like: A = memory[0x0F]

```
LDA $0F
```

  - Transfer the value stored in `B` into `A`
    - This is like: A = B

```
TFR B,A
```

Opcodes

- The opcode names usually make sense once you know them (just like the `vim` command set)
  - `LDA` / `LDX` / `LDY`
    - Load a value from memory into the given register
  - `STA` / `STX` / `STY`
    - Store the value of the register in memory
  - `TFR`
    - Transfer from one register to another (`TFR B,A` Transfer from `B` to `A`)
- However, the hard part is the missing instructions
  - 6809 has no `DIV` or `SQRT`

6809 Mnemonic Opcodes

Code	Meaning	Code	Meaning	Code	Meaning	Code	Meaning
ABX	Add to Index Register	BPL m	Branch if Plus	JSR s	Jump to Subroutine	ROL d	Rotate Left
ADCa s	Add with Carry	BRA m	Branch Always	LBcc nn	Branch	ROLa	Rotate Left acc.
ADDa s	Add	BRN m	Branch Never	LBRA nn	Long Branch Always	ROR d	Rotate Right
ADDD s	Add to Double acc.	BSR m	Branch to Subroutine	LBSR nn	Long Branch Subroutine	RORa	Rotate Right acc.
ANDa s	Logical AND	BVC m	Branch if Overflow Clr	LDa s	Load accumulator	RTI	Return from Interrupt
ANDCC s	Logical AND with CC	BVS m	Branch if Overflow Set	LDD s	Load Double acc.	RTS	Return from Subroutine
ASL d	Arithmetic Shift Left	CLR d	Clear	LDS s	Load Stack pointer	SBCa s	Subtract with Carry
ASLa	Arithmetic Shift Left	CLRa	Clear accumulator	LDU s	Load User stack ptr	SEX	Sign Extend
ASR d	Arithmetic Shift Right	CMPa s	Compare	LDi s	Load index register	STa d	Store accumultor
ASRa	Arithmetic Shift Right	CMPD s	Compare Double acc.	LEAp s	Load Effective Address	STD d	Store Double acc.
BCC m	Branch if Carry Clear	CMPS s	Compare Stack pointer	LSL d	Logical Shift Left	STS d	Store Stack pointer
BCS m	Branch if Carry Set	CMPU s	Compare User stack ptr	LSLa	Logical Shift Left	STU d	Store User stack ptr
BEQ m	Branch if Equal	CMPi s	Compare	LSR d	Logical Shift Right	STi d	Store index register
BGE m	Branch if Great/Equal	COM d	Complement	LSRa	Logical Shift Right	SUBa s	Subtract
BGT m	Branch if Greater Than	COMa	Complement accumulator	MUL	Multiply	SUBD s	Subtract Double acc.
BHI m	Branch if Higher	CWAI n	Wait for int.	NEG d	Negate	SWI	Software Interrupt 1
BHS m	Branch if Higher/Same	DAA	Decimal Adjust Acc.	NEGa	Negate accumulator	SWI2	Software Interrupt 2
BITa s	Bit Test accumulator	DEC d	Decrement	NOP	No Operation	SWI3	Software Interrupt 3
BLE m	Branch if Less/Equal	DECa	Decrement accumulator	ORa s	Logical inclusive OR	SYNC	Sync. to interrupt
BLO m	Branch if Lower	EORa s	Logical Exclusive OR	ORCC n	Inclusive OR CCR	TFR r,r	Transfer
BLS m	Branch if Lower/Same	EXG r,r	Exchange	PSHS r	Push reg(s) (not S)	TST s	Test
BLT m	Branch if Less Than	INC d	Increment	PSHU r	Push reg(s) (not U)	TSTa	Test accumulator
BMI m	Branch if Minus	INCa	Increment accumulator	PULS r	Pull reg(s) (not S)		
BNE m	Branch if Not Equal	JMP s	Jump	PULU r	Pull reg(s) (not U)		

Functional Groups

- Instructions exist for
  - Data transfer
    - Transfer data between registers or registers and memory
    - Push and pull from the stack
  - Data processing
    - Arithmetic, logical, shift, operations
  - Test and branch
    - Check bits in the flag (`CC`) register and conditional jump
  - Input / output
    - Fast Instructions that access the `DP` page
    - `OUT` and `IN` instructions on x86
  - Control
    - Interrupt handling

Data Transfer Instructions

- Memory to registers
  - `LDA` Load register
  - `LDB`
  - `LDD`
  - `LDS`
  - `LDU`
  - `LDX`
  - `LDY`
- Register to memory
  - `STA` Store register
  - `STB`
  - `STD`
  - `STS`
  - `STU`
  - `STX`
  - `STY`
- Between registers
  - `TFR` Transfer register to register
  - `EXG` Exchange registers
- Inherent
  - `CLRA` Clear register
  - `CLRB`
  - `CLR`
  - `COMA` Complement (invert)
  - `COMB`
  - `COM`
- Stack
  - `PSHS` Push onto stack
  - `PSHU`
  - `PULS` Pull from stack (pop)
  - `PULU`

Data Processing Instructions

- Arithmetic
  - `ABX` Add B to X
  - `ADCA` Add with carry
  - `ADCB`
  - `ADDA` Add
  - `ADDB`
  - `ADDD`
  - `SBCA` Subtract with carry
  - `SBCB`
  - `SUBA` Subtract
  - `SUBB`
  - `SUBD`

- Arithmetic
  - `DECA` Decrement
  - `DECB`
  - `DEC`
  - `INCA` increment
  - `INCB`
  - `INC`
  - `LEAS` Load effective address
  - `LEAU`
  - `LEAX`
  - `LEAY`
  - `MUL` Multiply
  - `NEGA` Negate (2's complement)
  - `NEGB`
  - `NEG`

Data Processing Instructions

- Logic
  - `ANDA` And
  - `ANDB`
  - `ANDCC`
  - `ORA` Or
  - `ORB`
  - `ORCC`
  - `EORA` Exclusive or
  - `EORB`
- Shifts
  - `ASLA` Arithmetic shift left
  - `ASLB`
  - `ASL`
  - `ASRA` Arithmetic shift right
  - `ASRB`
  - `ASR`
  - `LSLA` Logical shift left
  - `LSLB`
  - `LSL`
  - `LSRA` Logical shift right
  - `LSRB`
  - `LSR`
  - `ROLA` Rotate left
  - `ROLB`
  - `ROL`
  - `RORA` Rotate right
  - `RORB`
  - `ROR`

Test and Jump Instructions

- Comparisons
  - `BITA` Test bit in accumulator
  - `BITB`
  - `CMPA` Compare register
  - `CMPB`
  - `CMPD`
  - `CMPS`
  - `CMPU`
  - `CMPX`
  - `CMPY`
  - `TSTA` Test (set CC) register
  - `TSTB`
  - `TST`
- Jumps
  - `JMP` Jump
  - `JSR` Jump to subroutine
  - `RTS` Return from subroutine

Branch Instructions

- Branches
  - `BCC` Branch carry clear
  - `BCS` Branch carry set
  - `BEQ` Branch equal
  - `BGE` Branch greater or equal
  - `BGT` Branch greater
  - `BHI` Branch higher
  - `BHS` Branch higher or same
  - `BLE` Branch less or equal
  - `BLO` Branch lower
  - `BLS` Branch lower or same
  - `BLT` Branch less than
  - `BMI` Branch is negative
  - `BNE` Branch not equal
  - `BPL` Branch is positive
  - `BRA` Branch always
  - `BRN` Branch never
  - `BSR` Branch to subroutine
  - `BVC` Branch overflow clear
  - `BVS` Branch overflow set
- Long Branches
  - `LBCC` Branch carry clear
  - `LBCS` Branch carry set
  - `LBEQ` Branch equal
  - `LBGE` Branch greater or equal
  - `LBGT` Branch greater
  - `LBHI` Branch higher
  - `LBHS` Branch higher or same
  - `LBLE` Branch less or equal
  - `LBLO` Branch lower
  - `LBLS` Branch lower or same
  - `LBLT` Branch less than
  - `LBMI` Branch is negative
  - `LBNE` Branch not equal
  - `LBPL` Branch is positive
  - `LBRA` Branch always
  - `LBRN` Branch never
  - `LBSR` Branch to subroutine
  - `LBVC` Branch overflow clear
  - `LBVS` Branch overflow set

Control Instructions

- Control
  - `DAA` Decimal adjust
  - `SEX` Sign extend A into D
- Interrupts
  - `CWAI` Wait for interrupt
  - `SYNC` Sync to interrupt
  - `SWI` Software interrupt (system call)
  - `SWI2`
  - `SWI3`
  - `RTI` Return from interrupt
- No op
  - `NOP` No operation (do nothing)

Homework

- PLEASE:
  - Become familiar with the names of the 6809 registers
  - Read through the opcode list and become familiar with it

- Use the internet to find out:
  - What is the purpose of the `DP` register?
  - What is the difference between the 6809 `BEQ` and `LBEQ`?
  - Hitachi made a 6809 variant called the 6309 (or HD6309). How does the 6309 differ from the 6809?